

MIGRATIONS NO LARAVEL

Prof João Siles

O que são Migrations?

Imagine que as Migrations são um **controle de versão para o seu banco de dados**, assim como o Git é para o seu código.

Em vez de enviar um arquivo .sql gigante para o seu colega, você cria arquivos PHP que descrevem como o banco deve ser. Quando o seu colega baixa o código novo, ele roda um comando e o Laravel cria as tabelas automaticamente para ele.

Por que usar?

- **Histórico:** Você consegue ver quem criou qual coluna e quando.
- **Padronização:** Todo o time trabalha com a mesma estrutura de banco.
- **Agilidade:** Você cria, altera ou deleta tabelas usando apenas comandos de terminal.

Como funciona na prática?

Cada migration é um arquivo dentro da pasta database/migrations. O nome do arquivo contém a data e a hora para o Laravel saber a ordem exata em que deve "rodar" cada uma.

A estrutura do arquivo

Toda migration tem dois métodos principais:

1. **up():** O que deve acontecer (criar uma tabela, adicionar uma coluna).
2. **down():** Como desfazer o que o up fez (excluir a tabela ou remover a coluna).

Mãos na massa: Criando sua primeira Migration

Vamos supor que queremos criar uma tabela de **Produtos**.

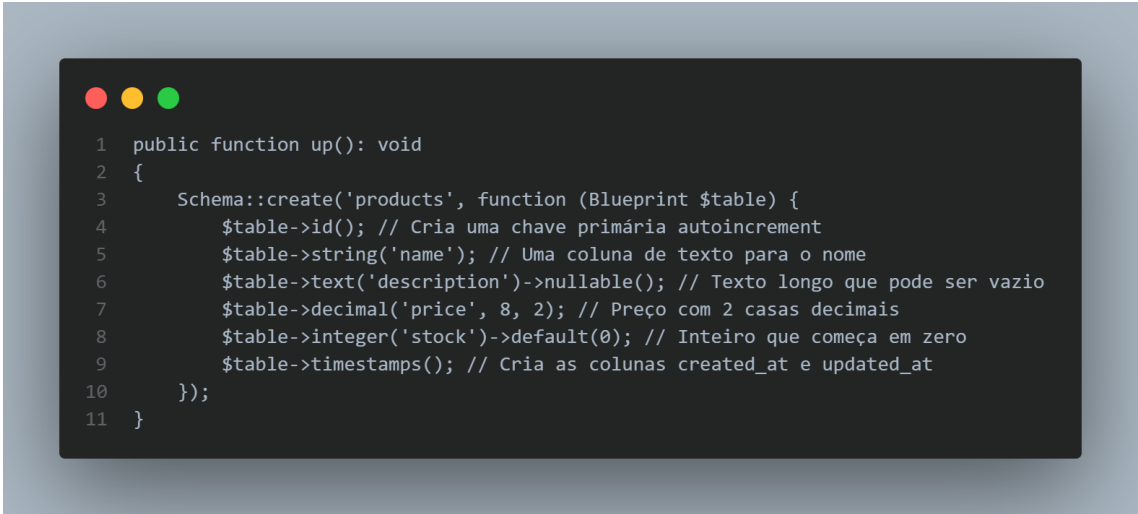
Passo 1: O comando

Abra o terminal na pasta do seu projeto e digite:

```
PS C:\Users\joaos\Desktop\projeto_laravel> php artisan make:migration create_products_table
```

Passo 2: Definindo os campos

O Laravel vai criar um arquivo. Abra-o e você verá algo assim dentro do método up:



```
1 public function up(): void
2 {
3     Schema::create('products', function (Blueprint $table) {
4         $table->id(); // Cria uma chave primária autoincrement
5         $table->string('name'); // Uma coluna de texto para o nome
6         $table->text('description')->nullable(); // Texto longo que pode ser vazio
7         $table->decimal('price', 8, 2); // Preço com 2 casas decimais
8         $table->integer('stock')->default(0); // Inteiro que começa em zero
9         $table->timestamps(); // Cria as colunas created_at e updated_at
10    });
11 }
```

Comandos Essenciais

Depois de escrever o código, você precisa avisar ao banco de dados para ler esses arquivos.

- **php artisan migrate:** Executa todas as migrations que ainda não foram rodadas. É o comando que você mais vai usar.
- **php artisan migrate:rollback:** Arrependeu? Esse comando desfaz a última ação (roda o método down).
- **php artisan migrate:status:** Mostra quais migrations já foram executadas e quais estão pendentes.

Exercício Prático

Agora é sua vez! Imagine que você está desenvolvendo um sistema para uma

Biblioteca.

Tarefa:

1. Crie uma migration chamada create_books_table.

2. A tabela deve ter os seguintes campos:

- title (texto)
- author (texto)
- isbn (texto, deve ser único)
- pages (número inteiro)
- is_available (booleano, com valor padrão "verdadeiro")

3. Tente rodar o comando no terminal para subir essa tabela.

Dica de ouro: Para campos únicos, usamos `$table->string('campo')->unique();` e para booleanos `$table->boolean('campo')->default(true);`.